Name: K.Likitha

Hall Ticket: 2303A52144

Batch: 41

Task 1: Refactoring Data Transformation Logic

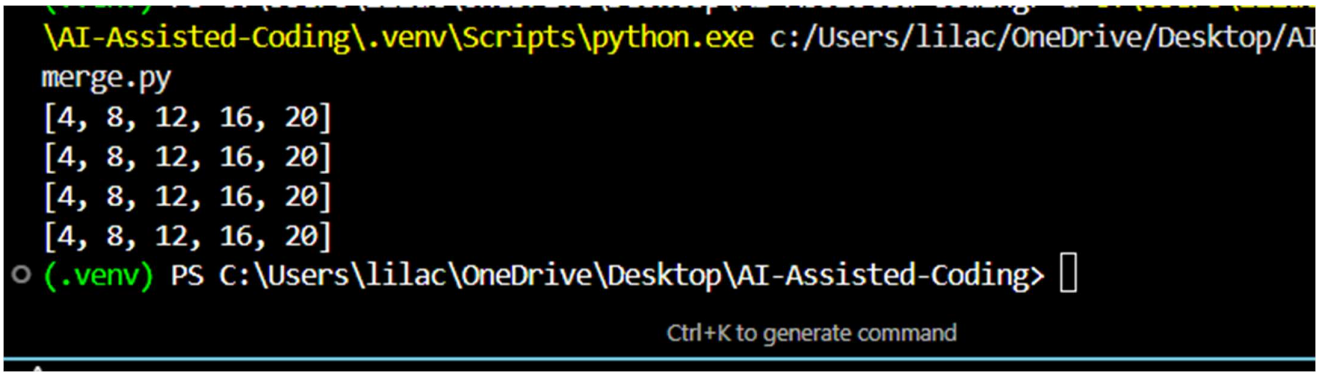
Prompt:

Refactor this loop into a more Python-style solution using list comprehension or a helper function, and make sure the output stays the same.

Code:

```
values = [2, 4, 6, 8, 10]
doubled = []
for v in values:
    doubled.append(v * 2)
print(doubled)
#Refactor this loop into a more Python-style solution using list comprehension or a helper
function.
doubled = [v * 2 for v in values]
print(doubled)
#Refactor this loop into a more Python-style solution using a helper function.
def double(v):
    return v * 2
doubled = list(map(double, values))
print(doubled)
#Refactor this loop into a more Python-style solution using a helper function.
def double(v):
    return v * 2
doubled = list(map(double, values))
print(doubled)
```

Output:



Explanation:

AI analyzes the loop pattern and replaces it with a shorter, more Pythonic structure like a list comprehension while keeping the output unchanged.

Task 2: **Improving Text Processing Code Readability**

Prompt:

Rewrite this string-building loop in a more readable and efficient Python way, keeping the final output unchanged.

Code:

```
words = ["Refactoring", "with", "AI", "improves","quality"]
message = ""
for w in words:
    message += w + " "
print(message.strip())
#Rewrite this string-building loop in a more readable and efficient Python way, keeping
the final output unchanged
message = " ".join(words)
print(message)
```

Output:

```
merge.py
Refactoring with AI improves quality
Refactoring with AI improves quality
(.venv) PS C:\Users\lilac\OneDrive\Desktop\AI-Assisted-Coding>
```

Explanation:

AI recognizes that repeated string concatenation inside a loop is inefficient and suggests using a cleaner method like " ".join(words) to improve readability and performance while keeping the output the same.

Task 3: Safer Access to Configuration Data

Prompt:

Refactor this manual dictionary key check into a safer and cleaner Python approach (such as using get), while keeping the same behavior for missing keys.

Code:

```
config = {"host": "localhost", "port": 8080}
if "timeout" in config:
    print(config["timeout"])
else:
    print("Default timeout used")
#Refactor this manual dictionary key check into a safer and cleaner Python approach, while
keeping the same behavior for missing keys.
if config.get("timeout") is not None:
    print(config["timeout"])
else:
    print("Default timeout used")
```

Output:

```
merge.py
Default timeout used
Default timeout used
(.venv) PS C:\Users\lilac\OneDrive\Desktop\AI-Assisted-Coding>
```

Explanation:

AI notices the manual key check pattern and suggests using a safer method like config.get() to avoid errors and make the code cleaner. It keeps the same behavior when the key is missing by allowing a default value.

Task 4: Refactoring Conditional Logic for Scalability

Prompt:

Refactor this multiple if–elif conditional logic into a cleaner and more scalable approach using a mapping technique, while keeping the same functionality.

Code:

```
action = "divide"
x, y = 10, 2
if action == "add":
    result = x + y
elif action == "subtract":
    result = x - y
elif action == "multiply":
    result = x * y
elif action == "divide":
    result = x / y
else:
    result = None
print(result)
#Refactor this multiple if-elif conditional logic into a cleaner and more scalable
approach using a mapping technique, while keeping the same functionality.
operations = {
    "add": lambda x, y: x + y,
    "subtract": lambda x, y: x - y,
    "multiply": lambda x, y: x * y,
    "divide": lambda x, y: x / y
}
result = operations.get(action, lambda x, y: None)(x, y)
print(result)
```

Output:

Explanation:

AI sees that many if–elif conditions make the code longer and harder to maintain, so it suggests using a dictionary mapping to make the logic cleaner and easier to extend in the future.

Task 5: Simplifying Search Logic in Collections

Prompt:

Refactor this manual loop-based search into a more concise and readable Python approach, while preserving the same output behavior.

Code:

```
inventory = ["pen", "notebook", "eraser", "marker"]
found = False
for item in inventory:
    if item == "eraser":
        found = True
        break
print("Item Available" if found else "Item Not Available")
#Refactor this manual loop-based search into a more concise and readable Python approach,
while preserving the same output behavior.
if "eraser" in inventory:
    print("Item Available")
else:
    print("Item Not Available")
```

Output:

```
  merge.py
  Item Available
  Item Available
(.venv) PS C:\Users\lilac\OneDrive\Desktop\AI-Assisted-Coding>
```

Explanation:

AI recognizes that manually looping to search an item is unnecessary and suggests using Python's built-in in operator to make the code shorter and easier to read while keeping the same result.